



Planner Web: Sơ Đồ Chi Tiết Của Một Hệ Thống Cộng Tác Hiện Đại

Một chuyến tham quan kiến trúc từ Client-side đến
Trợ lý AI thông minh.

Giải Pháp Toàn Diện Cho Không Gian Làm Việc Số

Planner Web là một hệ thống quản lý workspace và cộng tác đa chức năng, được xây dựng với kiến trúc Full-Stack hiện đại, cung cấp giải pháp toàn diện cho quản lý dự án, giao tiếp nhóm, và hỗ trợ AI.



Quản lý Workspace & Dự án

- ❏ Tạo và quản lý không gian làm việc.
- ❏ Quản lý tasks, lịch biểu, và timeline dự án.



Công cụ Cộng tác Real-time

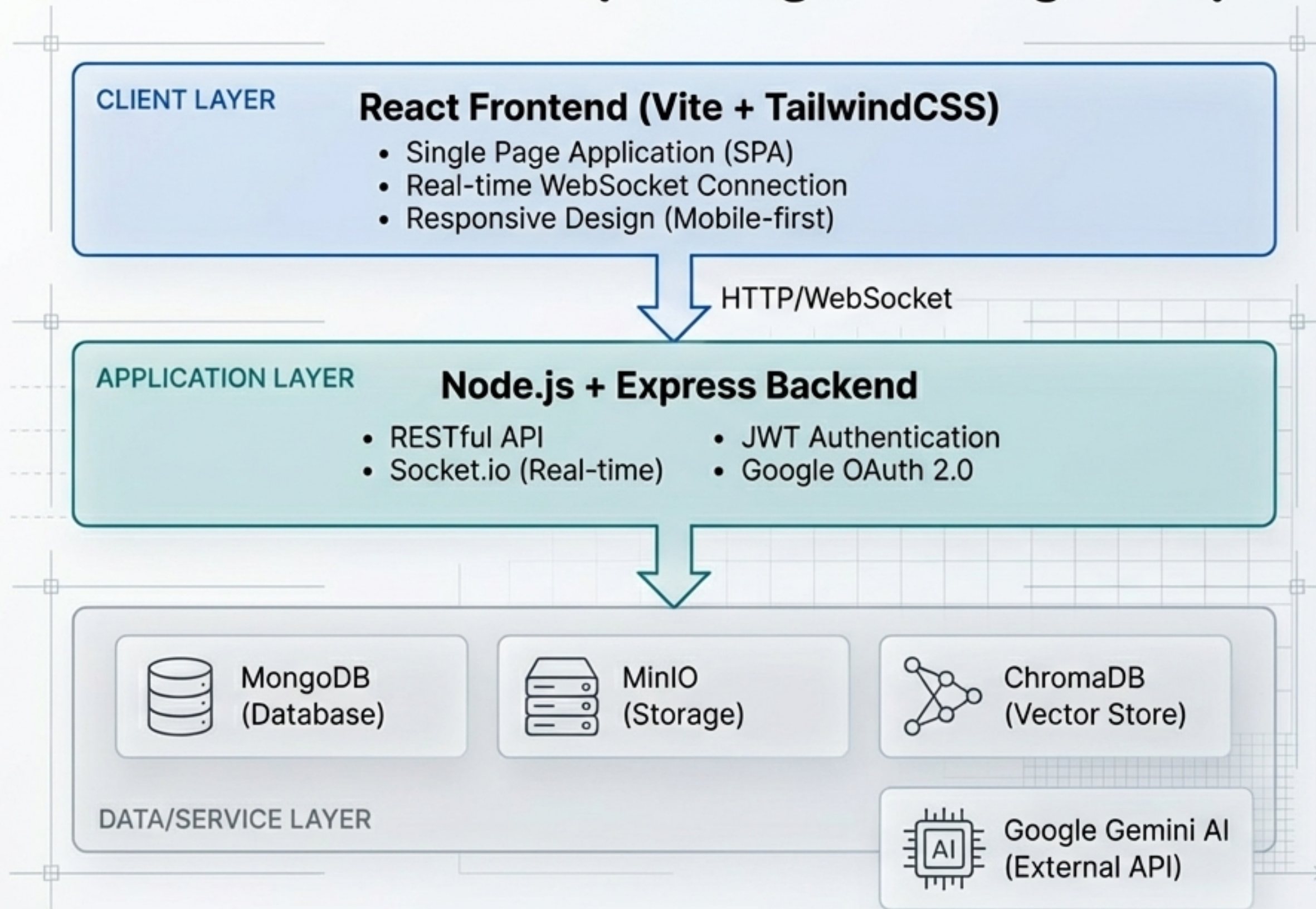
- ❏ Chat trực tiếp, video call, chia sẻ tài liệu.
- ❏ Giao tiếp tức thì qua WebSocket.



Quản lý Tài liệu & AI

- ❏ Lưu trữ tài liệu trên cloud (MinIO S3).
- ❏ Trợ lý AI thông minh với RAG hỗ trợ truy vấn tài liệu.

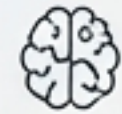
Sơ Đồ Kiến Trúc Hệ Thống: Ba Tầng Rõ Rệt



Bên Trong Backend: Một Kiến Trúc Logic và Mạch Lạc



config/ Cấu hình ứng dụng (Port, CORS, DB, Auth).



controllers/ Nơi xử lý Business Logic (e.g., ``authController``, ``workspaceController``).



models/ Định nghĩa Schema cho MongoDB (e.g., ``User.js``, ``Workspace.js``).



repositories/ Tầng Truy Cập Dữ Liệu (DAL) - Tách biệt logic DB.



services/ Tích hợp với các dịch vụ bên ngoài (MinIO, ChromaDB, Gemini).



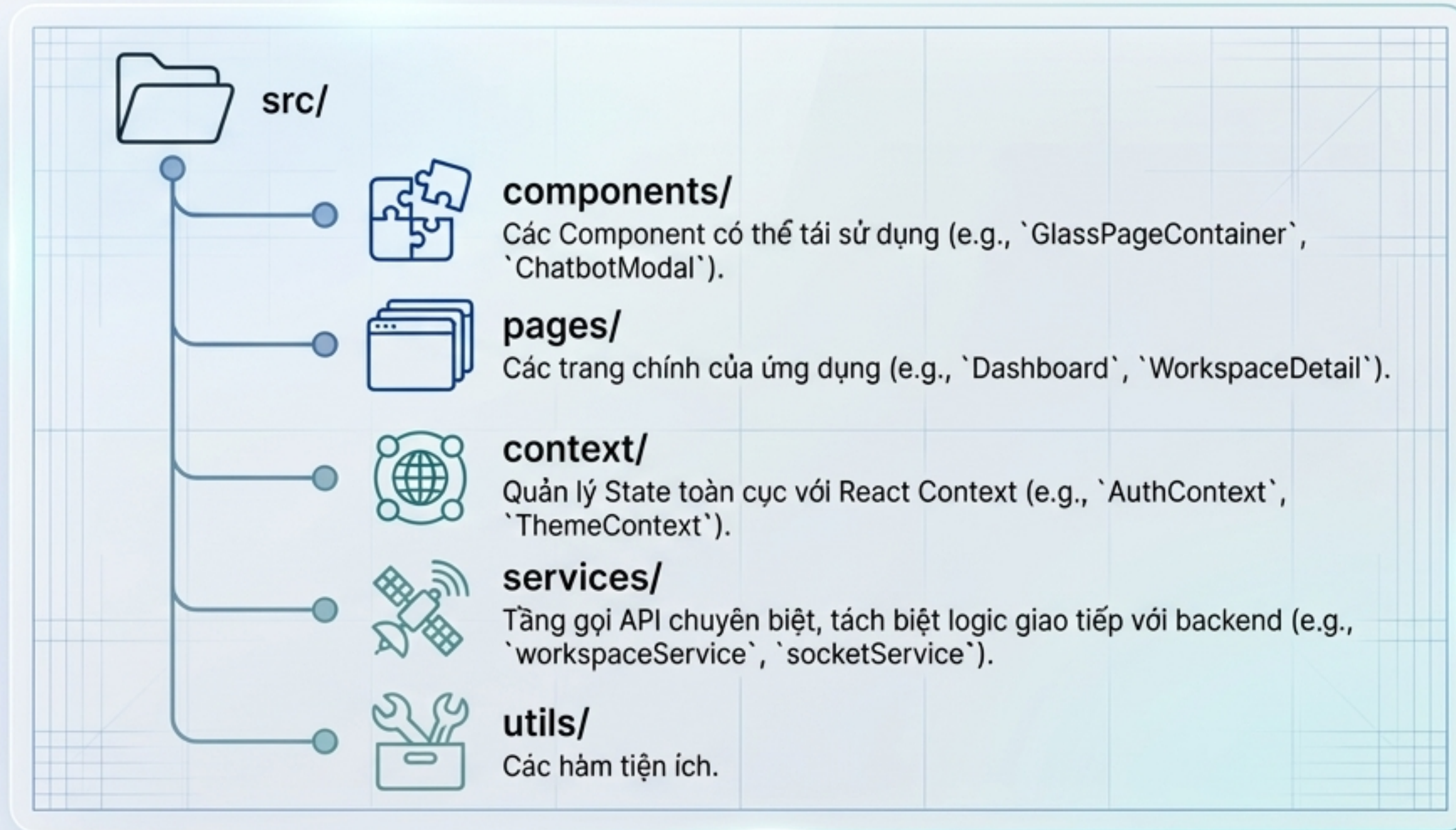
middlewares/ Logic trung gian (`Authentication`, `Security`, `Validation`).



routes/ Định tuyến các API Endpoints.

Cấu trúc module hóa cao, tuân thủ nguyên tắc Single Responsibility, giúp dễ dàng bảo trì và mở rộng.

Xây Dựng Trải Nghiệm Người Dùng: Cấu Trúc Frontend



Kiến trúc dựa trên component, quản lý state hiệu quả và một tầng service rõ ràng đảm bảo một codebase sạch sẽ và hiệu suất cao. Giao diện 'Glassmorphism' được xây dựng có chủ đích.

Nền Tảng Dữ Liệu: Mỗi Dịch Vụ Một Nhiệm Vụ



MongoDB (Database)

Purpose: Lưu trữ dữ liệu chính của ứng dụng (users, workspaces, messages).

Reason: Schema linh hoạt, phù hợp cho dữ liệu phi cấu trúc và phát triển nhanh.



MinIO (Object Storage)

Purpose: Lưu trữ các file media (tài liệu, avatar, file đính kèm).

Reason: Tương thích S3, self-hosted, hiệu quả cho việc lưu trữ và truy xuất file lớn.



ChromaDB (Vector Store)

Purpose: Lưu trữ vector embeddings từ tài liệu cho tính năng tìm kiếm ngữ nghĩa của AI.

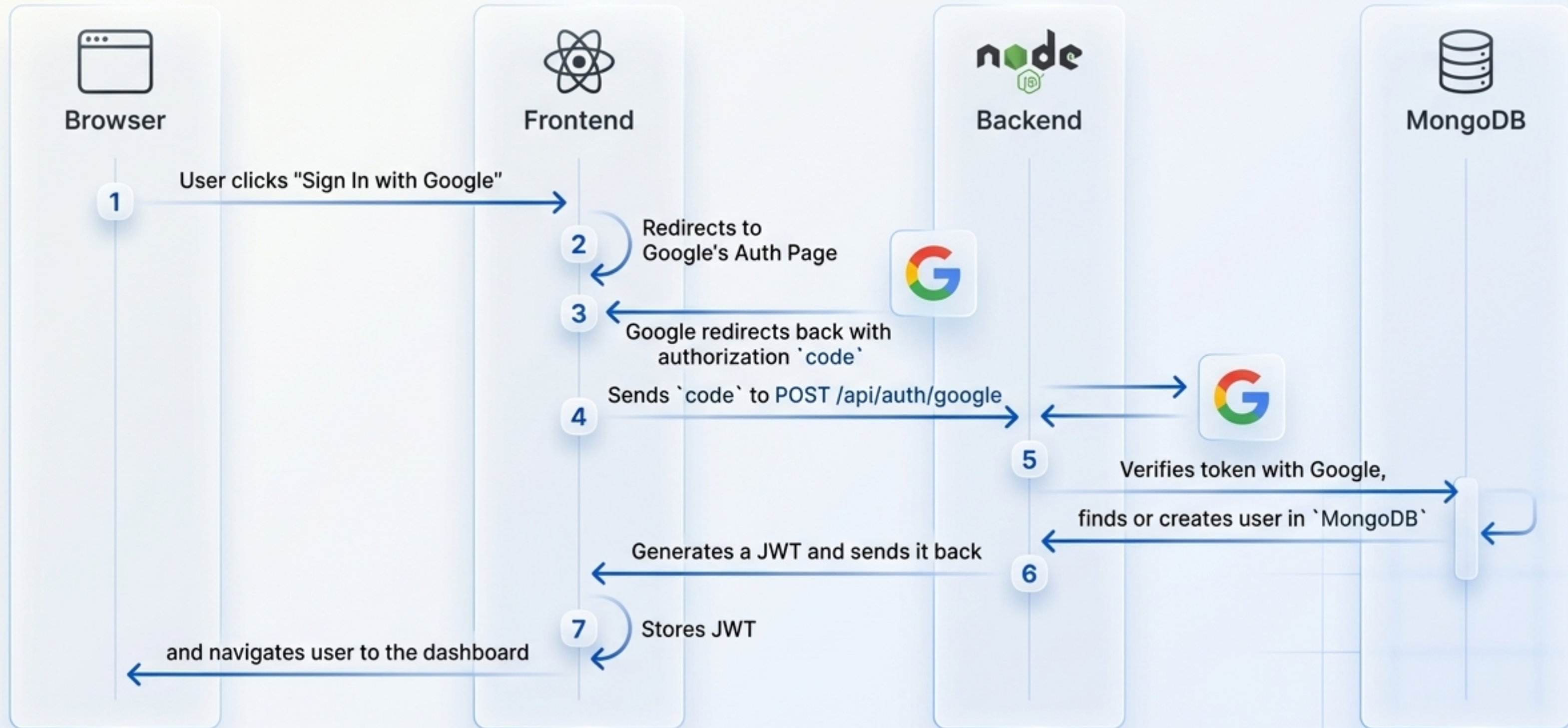
Reason: Chuyên dụng cho RAG, tối ưu hóa cho các truy vấn tương đồng vector.

Từ Sơ Đồ Tĩnh Đến Hệ Thống Sống: Các Luồng Hoạt Động Chính

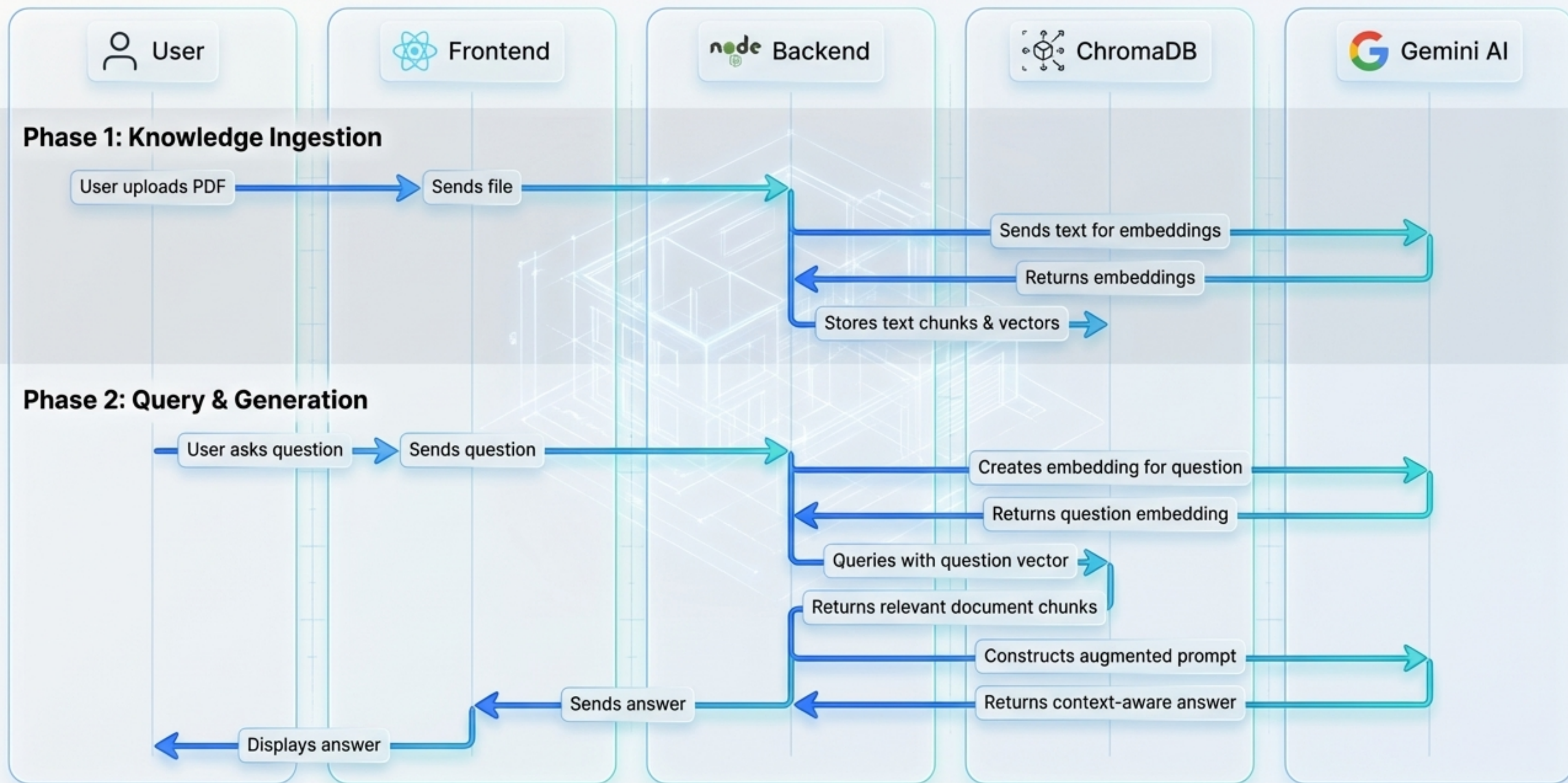
Kiến trúc được thiết kế để hỗ trợ các luồng tương tác phức tạp một cách liền mạch và hiệu quả. Hãy cùng xem cách các thành phần phối hợp với nhau trong các kịch bản quan trọng.



Luồng Xác Thực: Tích Hợp Google OAuth 2.0 An Toàn



Trí Tuệ Nhân Tạo: Luồng Chatbot với RAG (Retrieval-Augmented Generation)



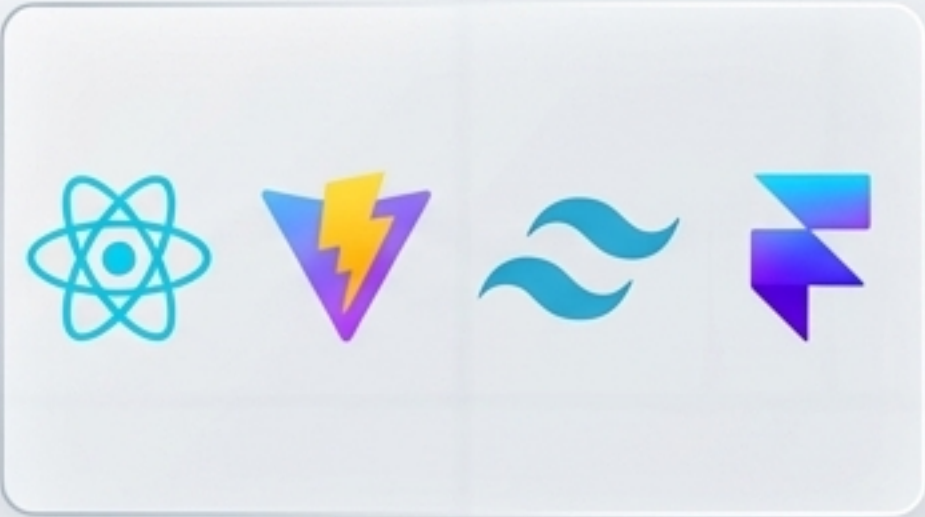
Bộ Công Cụ Công Nghệ (Tech Stack)

Backend Technologies



Inter Medium	Inter Regular
Mongoose	JWT
Helmet	Joi
Bcrypt	Axios

Frontend Technologies



Inter Medium	Inter Regular
React Router	Axios
Socket.io Client	Recharts
Heroicons	Moment.js

DevOps & Infrastructure



Inter Medium	Inter Regular
Docker	Containerization
Docker Compose	Orchestration
Jest	Testing
ESLint	Linting

Giao Diện Lập Trình Ứng Dụng (API)

RESTful API Endpoints

Authentication

`POST /api/auth/register`

`POST /api/auth/login`

`POST /api/auth/google`

Workspaces

`GET /api/workspaces`

Chatbot

`POST /api/chatbot/knowledge-base/upload`

`POST /api/chatbot/chat`

Documents

Messaging

`POST /api/chatbot/knowledge-base/upload`

Groups

Groups

`POST /api/chatbot/groups/chat`

WebSocket Events

Client to Server

`socket.emit('join-workspace', ...)`

`socket.emit('join-chat', ...)`

Server to Client

`socket.on('new-message', ...)`

`socket.on('workspace-updated', ...)`

API được thiết kế theo nguyên tắc REST, rõ ràng và dễ dàng tích hợp.

Xây Dựng Để Vận Hành: Bảo Mật, Kiểm Thử và DevOps



Security Features

- JWT Authentication & Refresh Tokens
- Password Hashing (Bcrypt)
- Helmet.js (Security Headers)
- CORS & Rate Limiting
- Input Validation (Joi)
- Mongoose Parameterized Queries (Chống SQL Injection)



Testing Strategy

Framework: Jest for both Backend & Frontend.

Commands: ``npm test``, ``npm test -- --coverage``

Focus: Unit & Integration tests for critical business logic.



Infrastructure & Deployment

Containerization: Toàn bộ hệ thống được đóng gói bằng **Docker**.

Orchestration: **Docker Compose** quản lý và khởi chạy tất cả các services (backend, frontend, databases) chỉ với một lệnh.

CI/CD: Tích hợp **GitHub Actions** cho quy trình tự động hóa.

Khởi Chạy Hệ Thống

1. Clone Repository

```
git clone https://github.com/your-repo/planner-web.git  
cd planner-web
```

2. Cấu hình Environment

- Tạo file `.env` cho `backend` và `frontend` từ các file mẫu.
- Điền các thông tin cần thiết: `MONGODB\_URI`, `JWT\_SECRET`, `GEMINI\_API\_KEY`, etc.

3. Khởi động với Docker

```
docker-compose up -d
```

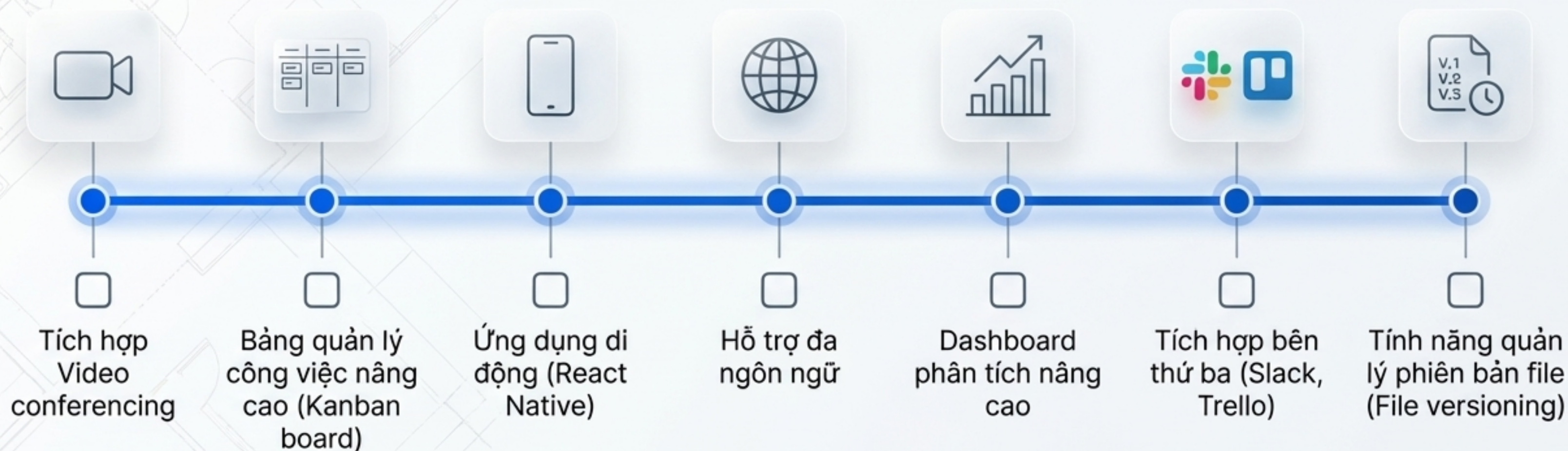
Access Points:

- **Frontend:** <http://localhost:3000>
- **Backend API:** <http://localhost:3001>
- **MinIO Console:** <http://localhost:9001>

Hệ thống có thể được triển khai cục bộ trong vài phút nhờ Docker.

Lộ Trình Phát Triển: Hướng Tới Tương Lai

Kế hoạch cho Phiên bản 2.0



Kiến trúc hiện tại được xây dựng với khả năng mở rộng, sẵn sàng cho những tính năng đột phá trong tương lai.